

Docker Deployment Guide

Complete guide for deploying the Property RAG System using Docker and Docker Compose.

Prerequisites

Docker Engine 20.10+ installed Docker Compose 2.0+ installed At least 4GB RAM available 5GB free disk space

Install Docker

Ubuntu/Debian: `bash curl -fsSL https://get.docker.com -o get-docker.sh sudo sh get-docker.sh sudo usermod -aG docker $USER`

macOS: `bash brew install --cask docker`

Windows: Download Docker Desktop from <https://www.docker.com/products/docker-desktop>

Quick Start

1. Clone or Navigate to Project

```
bash cd /path/to/Property_DataRAG_System
```

2. Set Environment Variables

```
```bash
```

## Copy example env file

```
cp .env.example .env
```

## Edit .env and add your API key (optional)

```
nano .env ```
```

Add your Gemini API key: `env GEMINI_API_KEY=your_api_key_here`

### 3. Build and Start Services

```
```bash
```

Build images and start all services

```
docker-compose up -d
```

View logs

```
docker-compose logs -f ```
```

4. Load Data (First Time Only)

```
```bash
```

## Execute data loading script inside container

```
docker-compose exec backend python scripts/load_data.py ```
```

#### 5. Access Application

**Backend API:** <http://localhost:8000> **API Docs:** <http://localhost:8000/docs> **Frontend UI:** <http://localhost:8501> **Health Check:** <http://localhost:8000/health>

## Docker Commands

### Start Services

```
```bash
```

Start in foreground

```
docker-compose up
```

Start in background

```
docker-compose up -d ```
```

Stop Services

```
```bash
```

## Stop services

```
docker-compose stop
```

## Stop and remove containers

```
docker-compose down
```

## Stop and remove containers + volumes

```
docker-compose down -v ```
```

### View Logs

```
```bash
```

All services

```
docker-compose logs -f
```

Specific service

```
docker-compose logs -f backend docker-compose logs -f frontend ```
```

Rebuild Images

```
```bash
```

## Rebuild all services

```
docker-compose build
```

## Rebuild specific service

```
docker-compose build backend
```

## Rebuild and restart

```
docker-compose up -d --build ```
```

## Execute Commands in Container

```
```bash
```

Backend shell

```
docker-compose exec backend bash
```

Run Python script

```
docker-compose exec backend python scripts/load_data.py
```

Run tests

```
docker-compose exec backend python tests/test_rag_pipeline.py ```
```

Architecture

Services

1. Backend Service - Image: Custom (built from Dockerfile) - Port: 8000 - Contains: FastAPI, ChromaDB, RAG pipeline - Persistent Storage: ChromaDB data, analytics logs

2. Frontend Service - Image: Python 3.11 slim - Port: 8501 - Contains: Streamlit UI - Connects to: Backend service

Volumes

yml volumes: - ./backend/chroma_db:/app/backend/chroma_db # Vector database -
./analytics:/app/analytics # Query logs

Network

Services communicate via Docker network property-rag-network.

Production Deployment

1. Update docker-compose.prod.yml

Create production compose file: ```yaml version: '3.8'

services: backend: build: . image: property-rag-backend:latest ports: - "8000:8000" environment: - GEMINI_API_KEY=\${GEMINI_API_KEY} volumes: - chroma_data:/app/backend/chroma_db - analytics_data:/app/analytics restart: always deploy: resources: limits: cpus: '2' memory: 4G reservations: cpus: '1' memory: 2G

frontend: image: property-rag-frontend:latest ports: - "8501:8501" environment: - BACKEND_URL=http://backend:8000 depends_on: - backend restart: always deploy: resources: limits: cpus: '1' memory: 1G

nginx: image: nginx:alpine ports: - "80:80" - "443:443" volumes: - ./nginx.conf:/etc/nginx/nginx.conf - ./ssl:/etc/nginx/ssl depends_on: - backend - frontend restart: always

volumes: chroma_data: analytics_data: ```

2. Deploy to Production

```bash

## Build images

docker-compose -f docker-compose.prod.yml build

## Start services

docker-compose -f docker-compose.prod.yml up -d

## Load data

docker-compose -f docker-compose.prod.yml exec backend python scripts/load\_data.py ```

# Cloud Deployment

## AWS ECS/Fargate

1. Build and Push to ECR: ```bash

## Login to ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
.dkr.ecr.us-east-1.amazonaws.com
```

## Build and tag

```
docker build -t property-rag-backend . docker tag property-rag-backend:latest
.dkr.ecr.us-east-1.amazonaws.com/property-rag-backend:latest
```

## Push

```
docker push .dkr.ecr.us-east-1.amazonaws.com/property-rag-backend:latest ```
```

2. Create ECS Task Definition: json { "family": "property-rag", "containerDefinitions": [ { "name":  
"backend", "image": "<account-id>.dkr.ecr.us-east-1.amazonaws.com/property-rag-backend:latest",  
"portMappings": [ { "containerPort": 8000, "protocol": "tcp" } ], "environment": [ { "name":  
"GEMINI\_API\_KEY", "value": "your-api-key" } ], "memory": 4096, "cpu": 2048 } ] }

## Google Cloud Run

```
```bash
```

Build and push to GCR

```
gcloud builds submit --tag gcr.io/PROJECT_ID/property-rag-backend
```

Deploy

```
gcloud run deploy property-rag-backend \ --image gcr.io/PROJECT_ID/property-rag-backend \ --platform  
managed \ --region us-central1 \ --allow-unauthenticated \ --set-env-vars GEMINI_API_KEY=your-key ```
```

Azure Container Instances

```
```bash
```

## Login to Azure

```
az login
```

## Create resource group

```
az group create --name property-rag-rg --location eastus
```

## Deploy container

```
az container create \ --resource-group property-rag-rg \ --name property-rag-backend \ --image
your-registry/property-rag-backend:latest \ --cpu 2 \ --memory 4 \ --ports 8000 \ --environment-variables
GEMINI_API_KEY=your-key ``
```

## Monitoring & Debugging

### Health Checks

```
``bash
```

## Check backend health

```
curl http://localhost:8000/health
```

## Check container health

```
docker-compose ps ``
```

### View Resource Usage

```
``bash
```

## Container stats

```
docker stats
```

## Specific service stats

```
docker stats property-rag-backend ``
```

### Debugging

```
``bash
```

## View container logs

```
docker-compose logs -f backend
```

## Execute bash in container

```
docker-compose exec backend bash
```

## Inspect container

```
docker inspect property-rag-backend ````
```

### Common Issues

Issue: Container exits immediately ````bash

## Check logs

```
docker-compose logs backend
```

## Common fix: Rebuild

```
docker-compose up -d --build ````
```

Issue: ChromaDB data not persisting ````bash

## Check volume mounts

```
docker-compose config
```

## Verify volume exists

```
docker volume ls ````
```

Issue: Port already in use ````bash

## Find process using port

```
lsof -i :8000
```

## Kill process or change port in docker-compose.yml

```
````
```

Scaling

Horizontal Scaling

```
````bash
```

## Scale backend to 3 instances

```
docker-compose up -d --scale backend=3
```

## Add load balancer (nginx)

```
docker-compose -f docker-compose.prod.yml up -d ``
```

### Resource Limits

Update docker-compose.yml: yml services: backend: deploy: resources: limits: cpus: '2' memory: 4G  
reservations: cpus: '1' memory: 2G

## Backup & Restore

### Backup Vector Database

```
``bash
```

## Backup ChromaDB data

```
docker-compose exec backend tar czf /tmp/chroma_backup.tar.gz backend/chroma_db docker cp
property-rag-backend:/tmp/chroma_backup.tar.gz ./backups/
```

## Alternative: Volume backup

```
docker run --rm -v property_rag_chroma_data:/data -v $(pwd)/backups:/backup \ ubuntu tar czf
/backup/chroma_backup.tar.gz /data ``
```

### Restore Vector Database

```
``bash
```

## Restore from backup

```
docker cp ./backups/chroma_backup.tar.gz property-rag-backend:/tmp/ docker-compose exec backend tar
xzf /tmp/chroma_backup.tar.gz -C / docker-compose restart backend ``
```

## Security Best Practices

### 1. Use Secrets Management

```
``yaml
```

## docker-compose.yml with secrets

```
services: backend: secrets: - gemini_api_key environment: -
GEMINI_API_KEY_FILE=/run/secrets/gemini_api_key
```

```
secrets: gemini_api_key: file: ./secrets/gemini_api_key.txt ```
```

## 2. Run as Non-Root User

Update Dockerfile: ```dockerfile

## Create non-root user

```
RUN useradd -m -u 1000 appuser USER appuser ```
```

## 3. Network Security

```
```yaml
```

Isolated backend network

```
networks: frontend: backend:
```

```
services: backend: networks: - backend frontend: networks: - frontend - backend ```
```

Performance Tuning

Optimize Image Size

```
```dockerfile
```

## Multi-stage build

```
FROM python:3.11-slim as builder WORKDIR /app COPY requirements.txt . RUN pip install --user -r requirements.txt
```

```
FROM python:3.11-slim COPY --from=builder /root/.local /root/.local ENV PATH=/root/.local/bin:$PATH COPY . . CMD ["python", "backend/main.py"] ```
```

### Cache Optimization

```
```bash
```

Build with cache

```
docker-compose build --parallel
```

Build without cache

```
docker-compose build --no-cache ```
```

Cleanup

Remove All Containers

```
```bash
```

## Stop and remove containers

```
docker-compose down
```

## Remove containers, networks, volumes

```
docker-compose down -v
```

## Remove images

```
docker rmi $(docker images -q property-rag-*) ```
```

## System Cleanup

```
```bash
```

Remove unused containers, networks, images

```
docker system prune -a
```

Remove volumes

```
docker volume prune ```
```

Next Steps

Set up monitoring: Integrate Prometheus & Grafana **Add CI/CD:** GitHub Actions or GitLab CI **Configure SSL:** Use Let's Encrypt with Nginx **Set up logging:** ELK stack or CloudWatch **Implement caching:** Redis for query caching

For production deployment support, refer to: - AWS ECS Documentation - Google Cloud Run Guide - Azure Container Instances